

Sentiment Analysis Using TF-IDF and Logistic Regression

Soumyabrata Bhat

1 Overview

This system classifies short text into one of three sentiment classes — *positive*, *negative*, or *neutral* — using a supervised machine learning pipeline. The model is trained on 3 000 labelled sentences from Amazon product reviews, IMDb movie reviews, and Yelp restaurant reviews (the UCI Sentiment Labelled Sentences dataset [1]).

The pipeline consists of three stages:

1. Vectorize raw text into numeric feature vectors (TF-IDF).
2. Classify the vector using Logistic Regression.
3. Apply a confidence gate to produce the final label.

2 Stage 1: TF-IDF Vectorization

TF-IDF (Term Frequency – Inverse Document Frequency) converts a raw sentence into a sparse, L2-normalized numerical vector. Each dimension corresponds to a unigram or bigram from a vocabulary of the $V = 5\,000$ most frequent terms in the training corpus.

2.1 Term Frequency

For a term t appearing in document d , the term frequency is represented by its raw occurrence count in the document:

$$\text{tf}(t, d) = f_{t,d}.$$

2.2 Inverse Document Frequency

To downweight non-discriminative terms (e.g. “the”, “for”), the inverse document frequency (IDF) is calculated. Scikit-learn implements a smoothed formulation by default (`smooth_idf=True`) using the natural logarithm:

$$\text{idf}(t) = \ln \left(\frac{1 + N}{1 + \text{df}(t)} \right) + 1,$$

where N is the total number of documents in the training dataset and $\text{df}(t)$ is the number of documents containing term t .

2.3 L2 Vector Normalization

The raw TF-IDF score for term t_j in document d is:

$$v_{dj} = \text{tf}(t_j, d) \times \text{idf}(t_j).$$

To account for varying document lengths, scikit-learn performs cosine (L2) normalization to project the raw vector $\mathbf{v}_d = (v_{d1}, v_{d2}, \dots, v_{dV})^\top$ onto a unit hypersphere. The final normalized feature vector $\mathbf{x}_d$ is:

$$\mathbf{x}_d = \frac{\mathbf{v}_d}{\|\mathbf{v}_d\|_2} \quad \text{where} \quad \|\mathbf{v}_d\|_2 = \sqrt{\sum_{j=1}^V v_{dj}^2}.$$

3 Stage 2: Logistic Regression

Logistic Regression is a linear model that predicts the posterior probabilities of the binary target classes: $y \in \{\text{pos}, \text{neg}\}$.

3.1 Decision Boundary

The model learns a single weight vector $\mathbf{w} \in \mathbb{R}^V$ and a bias scalar b . The linear log-odds score z for input vector $\mathbf{x}$ is:

$$z = \mathbf{w}^\top \mathbf{x} + b.$$

3.2 Sigmoid Activation Function

The predicted probability of the positive class (p_{pos}) is modeled using the sigmoid function:

$$p_{\text{pos}} = P(y = \text{pos} \mid \mathbf{x}) = \frac{1}{1 + \exp(-z)},$$

and the negative probability (p_{neg}) is modeled as its complement:

$$p_{\text{neg}} = P(y = \text{neg} \mid \mathbf{x}) = 1 - p_{\text{pos}}.$$

3.3 Optimization and Regularization

During training, the model minimizes the L2-regularized negative log-likelihood (binary cross-entropy loss). With scikit-learn's default regularization parameter $C = 1.0$, the optimization objective is:

$$\min_{\mathbf{w}, b} \frac{1}{2} \mathbf{w}^\top \mathbf{w} + C \sum_{i=1}^N \ln \left(1 + \exp \left(-y^{(i)} z^{(i)} \right) \right),$$

where $y^{(i)} \in \{-1, 1\}$ is the ground-truth label for the i -th sentence, and optimization is carried out by the L-BFGS solver.

3.4 Compound Score

During inference, the continuous compound score c is computed directly as the difference between the class probabilities:

$$c = p_{\text{pos}} - p_{\text{neg}} \in [-1, 1].$$

4 Stage 3: Confidence Gate

The dataset contains only positive and negative labels. To produce a neutral output for ambiguous or low-confidence inputs, a heuristic threshold is applied:

$$\hat{y} = \begin{cases} \text{pos} & \text{if } \max(p_{\text{pos}}, p_{\text{neg}}) \geq 0.6 \text{ and } p_{\text{pos}} > p_{\text{neg}}, \\ \text{neg} & \text{if } \max(p_{\text{pos}}, p_{\text{neg}}) \geq 0.6 \text{ and } p_{\text{neg}} > p_{\text{pos}}, \\ \text{neu} & \text{otherwise (uncertain).} \end{cases}$$

If neither class reaches 60% confidence, the model considers the input ambiguous and labels it neutral. This adds a third class without requiring neutral training examples.

5 Complete Pipeline

The end-to-end flow for classifying a single raw input string s is:

$$s \xrightarrow{\text{TF-IDF}} \mathbf{x} \xrightarrow{\mathbf{w}^\top \mathbf{x} + b} z \xrightarrow{\text{sigmoid}} p_{\text{pos}}, p_{\text{neg}} \xrightarrow{\text{gate}} \hat{y}, c.$$

The model returns the calculated values for p_{pos} and p_{neg} , while setting the output schema's neutral score field $\text{neu} = 0.0$, letting the classification label $\hat{y}$ handle the neutral assignment when confidence falls below 60%.

The implementation is a single Python module using `scikit-learn` [2]. The trained pipeline (TF-IDF $\rightarrow$ LogisticRegression) is serialized implicitly by being reconstructed on every server start.

References

- [1] D. Kotzias et al., “From Group to Individual Labels using Deep Features,” *KDD* 2015.
<https://archive.ics.uci.edu/ml/datasets/Sentiment+Labelled+Sentences>
- [2] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *JMLR* 12, pp. 2825–2830, 2011.
<https://scikit-learn.org>